

CardioIA — Fase 3 — Relatório Parte 1 (Edge)

Disciplina / projeto: CardioIA — protótipo IoT de monitoramento cardíaco simulado

Escopo: Armazenamento e processamento no dispositivo (ESP32), fluxo de funcionamento e **lógica de resiliência** à perda de conectividade

Referências técnicas no repositório: `esp32/src/main.cpp`, `esp32/diagram.json`, `esp32/readme.md`

1. Objetivo deste relatório

Descrever, de forma objetiva, **como o firmware no edge opera** no cenário Wokwi/ESP32: captura dos sensores, formação das amostras, enfileiramento local e política de envio quando a “rede” está disponível ou indisponível — incluindo o que acontece quando a fila atinge capacidade máxima.

2. Visão do fluxo de funcionamento

O programa executa um laço contínuo (`loop`) em que três eixos avançam em paralelo:

- Entrada do usuário (BPM simulado):** dois botões com *pull-up* interno (`GPIO 4` aumenta, `GPIO 5` diminui) atualizam um contador de toques válidos (com *debounce*). O valor publicado como `bpm_sim` é $60 + 10 \times (\text{número de toques})$, limitado a um teto configurável, de modo a representar batimentos por minuto de forma simples para demonstração.
- Simulação de conectividade:** uma variável booleana (`g_wifiSimConnected`) é alterada **manualmente** com um botão no **GPIO 18** (no Wokwi: **Wi-Fi sim**, tecla `w`): **cada toque alterna** entre “online simulado” e “offline simulado”. Isso não substitui o estado real do `WiFi` do ESP32, mas **controla se o firmware tenta drenar a fila** para o mecanismo de transporte (MQTT quando habilitado, ou Serial como alternativa de laboratório). O Serial imprime uma linha a cada mudança para facilitar gravação e relatório.
- Amostragem periódica e fila:** a cada intervalo configurável (`SAMPLE_INTERVAL_MS`, atualmente **1 s** no código), o DHT22 é lido (temperatura e umidade). Monta-se uma estrutura `Sample` (`ts_ms`, `temp_c`, `hum_pct`, `bpm_sim`, flag de leitura válida) e ela é **sempre enfileirada**, independentemente da fase online/offline da simulação — garantindo que **a coleta não para** quando a “rede simulada” cai.

Em síntese: **coletar sempre** → **armazenar na fila** → **transmitir e esvaziar a fila apenas quando a política de “online simulado” e o transporte permitirem**.

3. Sensores no edge

Papel	Implementação	Observação
Obrigatório	DHT22 no <code>GPIO 15</code>	Um sensor físico, duas grandezas (<code>temp_c</code> , <code>hum_pct</code>).
Segundo sensor	Dois push-buttons (<code>GPIO 4</code> / <code>GPIO 5</code>)	Proxy educacional de BPM; ajuste fino por toques sem custo de hardware extra.
Controle de demo	Push-button Wi-Fi sim (<code>GPIO 18</code>)	Alterna manualmente a fase “rede ok” / “rede cortada” para demonstrar fila offline.

O diagrama Wokwi (`esp32/diagram.json`) espelha essas ligações para reprodutibilidade da demonstração.

4. Lógica de resiliência

4.1 Comportamento quando “offline” (simulado)

Enquanto `g_wifiSimConnected` é **falso**, a função de dreno **não executa**: nenhuma amostra é retirada da fila para MQTT/Serial. As amostras **continuam a ser geradas e empilhadas** no buffer local. Assim, o sistema simula o requisito de **continuar coletando sem conexão** e acumular pendências para envio posterior.

4.2 Comportamento quando “online” (simulado)

Com `g_wifiSimConnected` **verdadeiro**, o firmware tenta o transporte:

- **Com MQTT habilitado** (`secrets.h` presente): se `WiFi` e broker estiverem disponíveis, a fila é esvaziada com um `publish` **por amostra** (JSON por linha de telemetria). Se o `WiFi` não conectar (por exemplo, simulador sem *IoT Gateway*), o código pode **despejar a fila no Serial** como saída observável — alinhado à alternativa de laboratório sugerida no material (resiliência visível no monitor).
- **Sem MQTT** (build só Serial): na fase online simulada, a fila é drenada para o **Serial** em formato JSON.

O importante para o relatório de **resiliência** é: **pendências acumuladas na janela offline são consumidas na volta ao online**, desde que o transporte subjacente responda.

4.3 Capacidade limitada e política ao encher (modelo de negócio / edge real)

A fila é implementada como **buffer circular em RAM** com capacidade `MAX_QUEUE` = **512** amostras. Se uma nova amostra chega com a fila cheia, **a amostra mais antiga é descartada** (avança-se a cabeça da fila antes de inserir a nova). Essa política reflete uma decisão de produto em monitoramento contínuo: **preservar o dado mais recente** quando o dispositivo não pode crescer memória indefinidamente, em vez de parar a coleta ou corromper o buffer.

No **Wokwi**, a RAM é volátil (reinício apaga a fila). O código prevê **SPIFFS** como opcional em hardware físico; no simulador web o foco é demonstrar **lógica** de fila e dreno, não persistência em flash.

4.4 Sincronização e ordem

Cada amostra carrega `ts_ms` baseado em `millis()` para **ordem relativa** entre pontos. Em produção, recomendaria-se **NTP** ou carimbo no broker; para o escopo da disciplina, o essencial é que a **sequência de sincronização** (offline acumula → online drena) fique explícita no comportamento do firmware.

5. Conclusão (Parte 1)

O fluxo do edge do CardioIA cumpre o roteiro pedagógico da Parte 1: **dois sensores** (DHT22 + botões), **armazenamento local** via fila limitada em RAM, **Wi-Fi simulado por variável booleana** com **controle manual** (botão no diagrama) para alternar online/offline, e **resiliência** materializada na continuidade da coleta, no acúmulo de pendências e no esvaziamento controlado ao restabelecer a fase “online” — com política clara de **capacidade máxima** e **descarte das amostras mais antigas** quando o limite é atingido.

6. Evidências associadas (checklist da entrega)

- **Link do projeto Wokwi:** *(inserir URL pública do projeto ao publicar)*
- **Código C++ comentado:** `esp32/src/main.cpp` e projeto PlatformIO em `esp32/`